

string.h i obsługa argumentów z command-line

1. Napisać prosty program `hello.c`, który pobierze z wiersza poleceń (za pomocą `int argc` oraz `char* argv[]`) argumenty wejściowe od użytkownika:

1. powitanie

2. liczbę powtórzeń wyświetlenia powitania.

- o na początku: wyświetlić na ekranie liczbę argumentów, jaką podał użytkownik (zmienna `argc`)
- o potem wyświetlić na ekranie wszystkie wartości w tablicy `argv`, od indeksu `0` do `argc`
- o zbudować, skompilować i uruchomić z wiersza poleceń, z podanymi argumentami:
`.\hello.exe hi 34` a potem bez `.\hello.exe`.
- o potem, piszemy właściwy program:
 - sprawdzający, czy wejściowe argumenty podano poprawnie. Jeśli podano ich za mało bądź za dużo (`argc!=3`), wyświetlić na ekranie pomocną wiadomość i zakończyć działanie wcześniej, zakończyć działanie z kodem `1` (`return 1`)

```
USAGE: .\hello.exe twoje_powitanie liczba_powtorzen
```

- jeśli `argc==3`: program ma wyświetlić na ekranie powitanie od użytkownika (`argv[1]`) określoną przez niego liczbę razy (`argv[2]`) i zakończyć pracę z kodem `0` (`return 0`). **Tip:** użyj funkcji `atoi()` aby przekonwertować `argv[2]` na typ `int`.

2. Napisać program `add.exe`, który pobierze od użytkownika dwie liczby rzeczywiste z wiersza poleceń i wyświetli ich sumę z dokładnością do 5 miejsc po przecinku, na zasadzie:

```
.\add.exe 3.14 -4.5
3.14000 + (-4.50000) = -1.36000
.\add.exe 2.1 + 1.1
2.10000 + 1.10000 = 3.20000
```

- wygodnie użyć funkcji `atof()` z biblioteki `<stdlib.h>`, aby przekonwertować wartości z tablicy `argv` z formatu string na `double`.
- **UWAGA:** dobrze by było, żeby w przypadku, gdy druga podana liczba jest ujemna, jak w powyższym pierwszym przykładzie, aby wyświetlała się pomiędzy nawiasami. Wystarczy

prosty `if`.

3. Zrobić prosty kalkulator, `calc.exe`, który pobierze od użytkownika dwie liczby rzeczywiste oraz znak określający, jaki rodzaj działania chcemy wykonać: `+`, `-`, `/`, `x`. Na ekranie wyświetlić wynik. Powinno to działać tak:

```
.\calc.exe 3.14 - 3.0
3.14000 - 3.00000 = 0.14000
.\calc.exe 2 x 0
2 x 0 = 0.00000
.\calc.exe -5 - -4
-5.00000 - (-4.00000) = -9.00000
```

- **Tip:** program ma przyjąć 3 argumenty, środkowy to łańcuch znaków, który określa rodzaj działania, jakie musimy wykonać. Przyda się funkcja `strcmp` z `<string.h>`, do porównywania łańcuchów znaków, aby dopasować dane od użytkownika z jednym z 4 wariantów.
- **TIP2:** nie bez powodu, opcję mnożenia oznaczamy nie przez `*`, a `x`. Pobranie dosłownie `*` jako argument może być trudniejsze z wiersza poleceń, w zależności od systemu. Oczywiście jeśli program zostanie uruchomiony np. tak: `.\calc.exe 3 x 4`, to w ramach programu (kodu C) `3*4` nadal oznacza mnożenie i to jest działanie, jakie musimy wykonać i jego wynik wyświetlić.
- **Opcjonalnie rozszerzenie:** jeśli użytkownik poda tylko jeden argument `interactive`, to jest uruchomimy program tak: `.\calc.exe interactive`, program ma działać w trybie interaktywnym: użyć pętli `do...while`, aby wyświetlić menu tekstowe, pobierać od użytkownika dane i wyświetlać wynik aż do momentu, gdy mu się znudzi:

```
Co chcesz zrobić?
1 - dodawanie
2 - odejmowanie
3 - mnożenie
0 - nic (koniec)
```

4. (*)Zdefiniuj przykładowy łańcuch zawierający za równo: znak newline `\n`, tab `\t` oraz kilka słów przedzielonych spacjami. Wyświetl go za pomocą `printf` i specyfikatora formatu `%s`. Zwróć uwagę na specjalne wyświetlanie tych znaków. Teraz napisz funkcję `void printWS(char* someString)`, która wypisze na ekranie wejściowy łańcuch znaków, ale wyświetlając specjalnie każdy biały znak:

- każdy znak newline `'\n'` ma być wyświetlony jako `[_NEWLINE_]`

- o każdy tab `'\t'` ma być wyświetlony jako `[_TAB_]`
- o każda spacja ma być wyświetlona jako `[_]`.
- o **tip:** najprościej to zrobić za pomocą pętli `for` oraz kilku instrukcji warunkowych, czy też `switch case`. Należy użyć funkcji `strlen` aby znaleźć długość wejściowego łańcucha znaków, aby wiedzieć, ile przejść ma wykonać pętla.
- o **tip2:** `someString[i]` wyświetlimy za pomocą `printf("%c", someString[i])`, natomiast wyświetlenie pozostałych możliwości ogarniemy prościej np. `printf("[_NEWLINE_]")`
- o **tip3:** w instrukcjach warunkowych należy porównać wartość pod `i`-tym indeksem (`someString[i]`) z zadanymi możliwościami. Pamiętaj, że `someString[i]` to `char`, tak samo `'\t'` to `char`, ale już `"\t"` to tablica znaków.
- o Przykład działania:

```
char TEXT[] = "cat\t had 3 legs,\nbut"
" it was\t fine.";
printWS(TEXT);
...
...po uruchomieniu
...
cat[_TAB_]had[_]3[_]legs,[_NEWLINE_]but[_]it[_]was[_TAB_]fine.
```

5. Za pomocą funkcji `strncpy` z `<string.h>`, przekopiować z jednego niepustego łańcucha znaków (tablicy `char`, np. `char str1[20]`), zawartość do drugiego łańcucha `str2` (drugiej tablicy `char`). Sprawdzić, jak się wyświetla `str2` za pomocą `printf("%s \n", str2)` oraz `str1`. Sprawdzić, co się stanie, jeśli rozmiar `str2` nie jest równy rozmiarowi `str1` (oba przypadki - krótszy/ dłuższy).
6. Zdefiniować łańcuch `mystr` o długości co najmniej 10 znaków. Używając funkcji `strncpy` z biblioteki `<string.h>`, przekopiować do innego łańcucha znaków `char A[6]` pierwsze 5 znaków `mystr`, a do drugiego `B[8]`: pierwsze 7 znaków. Z kolei do `char C[4]` skopiować 3 znaki z `mystr`, poczynając od indeksu 4 (5 znaku) - do tego można wykorzystać arytmetykę wskaźników (funkcja `strncpy` jako jeden z argumentów oczekuje adresu pierwszego elementu łańcucha. nic nie stoi na przeszkodzie, żeby zamiast tego podać adres 4-tego elementu tego łańcucha). **Do każdego z łańcuchów znaków `A,B,C`, dołożyć na koniec samemu znak `\0`, oznaczający koniec łańcucha:**
 1. Sprawdzić, jak się wyświetlają `A,B,C` (przy użyciu instrukcji `printf`) przed dołożeniem na koniec `\0`

2. oraz jak się wyświetlają po dołożeniu tego dodatkowego znaku.